

Otulia Web: Deep Dive - Chapter 4: Authentication & Authorization

4.1 The Security Paradigm

Otulia utilizes a modern, stateless security architecture based on **JWT (JSON Web Tokens)** and **Google OAuth 2.0**. This ensures that the server doesn't need to remember who is logged in (session-less), making it highly scalable and secure.

4.2 The Authentication Flow

4.2.1 Standard Registration (`auth.routes.js`)

When a user signs up, we never store their raw password. We use **Bcrypt** to "salt" and "hash" it.

```
router.post("/register", async (req, res) => {
  const { name, email, password } = req.body;

  // 1. Password Hashing: Turns "password123" into "a$2b$10$X..."
  const salt = await bcrypt.genSalt(10);
  const hashedPassword = await bcrypt.hash(password, salt);

  // 2. Database Creation
  const user = await User.create({ name, email, password: hashedPassword });

  // 3. Immediate Login: We issue a JWT right away so they don't have to login again
  const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET, {
    expiresIn: "7d" });

  res.status(201).json({ token, user });
});
```

4.2.2 Google OAuth Integration

We use a hybrid flow where the Frontend gets an `idToken` from Google, and the Backend verifies it.

```
router.post("/google-login", async (req, res) => {
  const { idToken } = req.body;

  // 1. Verify token with Google's public keys
  const ticket = await client.verifyIdToken({
    idToken,
    audience: process.env.GOOGLE_CLIENT_ID,
  });

  const { email, name, sub: googleId, picture } = ticket.getPayload();

  // 2. Upsert Logic: Find user by email OR googleId
  let user = await User.findOne({ $or: [{ email }, { googleId }] });
```

```

if (!user) {
  user = await User.create({ name, email, googleId, profilePicture: picture });
}

// 3. Issue Otulia's own JWT (The "Passport" for our specific API)
const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET);
res.json({ token, user });
});

```

4.3 Authorization & Role Gating

Not all users are equal. Otulia has three distinct levels: **User**, **Agent**, and **Admin**.

4.3.1 Role Hierarchy

- **user** : Can view assets, manage favorites, and buy/rent.
- **agent** : Can do everything a user can, PLUS manage their own inventory and leads.
- **admin** : Full system control, including verifying agents and viewing global analytics.

4.3.2 Protecting Admin Routes (`admin.routes.js`)

We use a secondary middleware called `adminCheck` to protect sensitive system routes.

```

const adminCheck = async (req, res, next) => {
  // 1. req.user was already populated by authMiddleware
  if (req.user.role !== 'admin') {
    // 2. Throw a 403 Forbidden if they aren't an admin
    return res.status(403).json({ error: "ACCESS_DENIED_ADMIN_ONLY" });
  }
  next(); // Proceed to the admin logic
};

// Usage in a route:
router.get("/stats", authMiddleware, adminCheck, async (req, res) => {
  // This code only runs if the user is authenticated AND is an admin
});

```

4.4 The Frontend Security layer (`AuthContext.jsx`)

The frontend is responsible for storing the token and including it in every API call.

4.4.1 Token Persistence

```

// On Load: Check if a token exists in the browser's storage
const [token, setToken] = useState(localStorage.getItem('token'));

// In every API call:
const response = await fetch('/api/auth/me', {
  headers: {

```

```
        'Authorization': `Bearer ${token}` // This is how the server knows who you are
    }
});
```

4.4.2 Session Recovery

If a user refreshes the page, the `AuthProvider` immediately calls `/api/auth/me` .

- **Success:** The user is logged back in automatically.
- **Failure (Token Expired):** The app calls `logout()` , clears the `localStorage` , and redirects to the Login page.

4.5 Security Best Practices Implemented

1. **Environment Isolation:** Secrets like `JWT_SECRET` are never hardcoded.
2. **Password Complexity:** Enforced at the model level (`minlength: 6`).
3. **Data Sanitization:** The `select("-password")` method is used in all user-fetching queries to ensure hashes are never sent to the frontend.
4. **Verification Locking:** Once an agent is "Verified," the backend locks their Name and Company Name fields. They cannot change their identity without contacting Support, preventing "Identity Swapping" scams.